Dynamic Programming

Paradigms

A **greedy algorithm** always makes the choice that looks best at the moment. That is, it makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.

A **divide-and-conquer** algorithm breaks the problem into subproblems; solves them; and combines the solutions to obtain a solution to the original problem.

A **dynamic programming** algorithm breaks the problem into a series of overlapping subproblems; combine solutions of smaller subproblems to obtain a solution to the original problem.

Rod-Cutting Problem

Input: A rod of length n inches and a table of prices p_i, for each i.

Question: find the maximum revenue r_n obtainable by cutting the rod into smaller pieces.

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

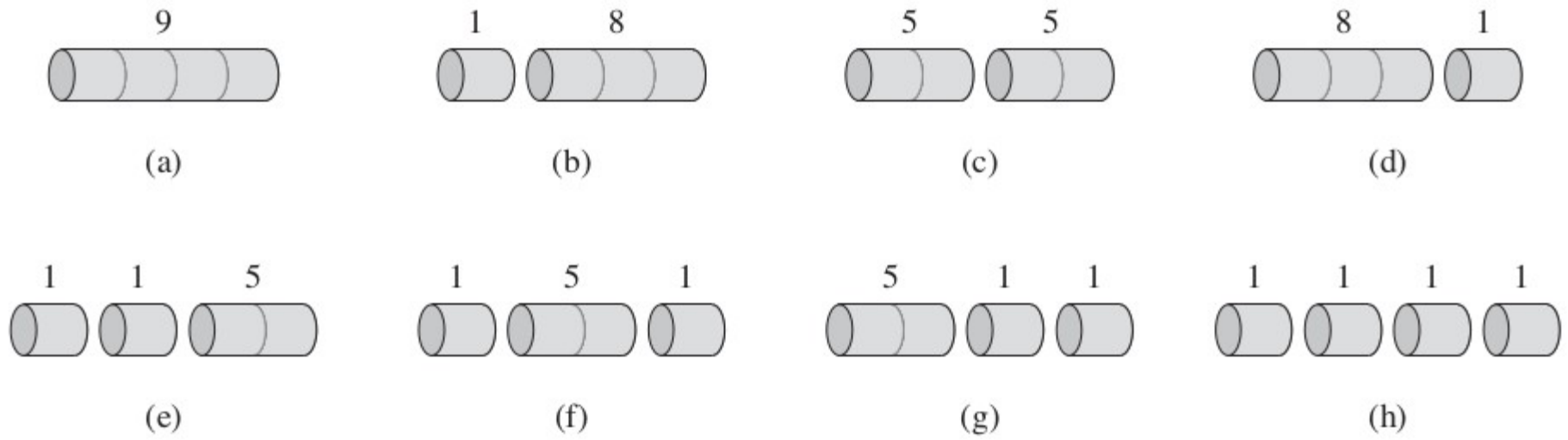
Rod of length 4

How many ways to cut it?

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

Rod of length 4

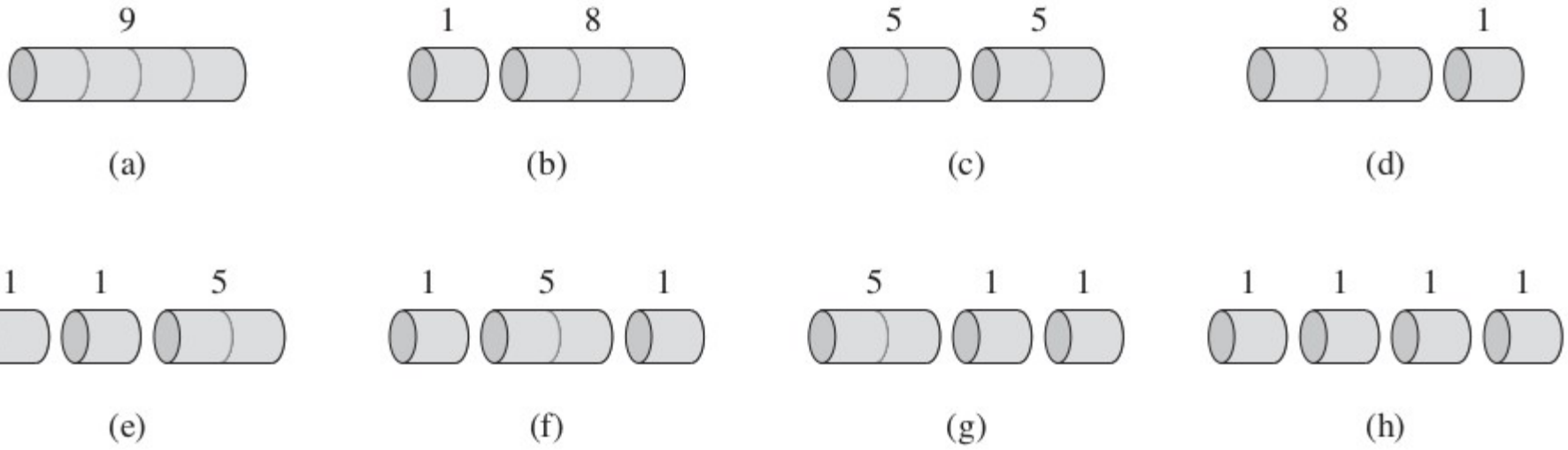
length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30



Rod of length 4

What is r_n?

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30



Understanding the problem

What is r_n?

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

Rod length	#cuts	Revenue (max)
1	0	1
2	0	5
3	0	8
4	1	10
5	1	13

Rod length	#cuts	Revenue (max)
6	0	17
7	1	18
8	1	22
9	1	25
10	0	30

Intuition

What is r_n?

#cuts?

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

Rod length	#cuts	Revenue (max)
1	0	1
2	0	5
3	0	8
4	1	10
5	1	13

Rod length	#cuts	Revenue (max)
6	0	17
7	1	18
8	1	22
9	1	25
10	0	30

Rod-Cutting Problem

Approach	Time
Brute Force	
Greedy	
Recursive algorithm	

Greedy Strategies

Approach - 1

Take the available length that has the highest cost.

Approach – 2

Take the ratio of p_i / i, and take the length i whose ratio is maximum.
If there are multiple such i, pick the i with the least value.

Approach 2

Take the ratio of p_i / i, and take the length i whose ratio is maximum.
If there are multiple such i, pick the i with the least value.

Rod length	Price P_i	Revenue (max)
1	1	1
2	5	5
3	8	8
4	9	10
5	10	13
6	17	17
7	17	18
8	20	22

Rod length	Revenue (max)	P_i / i
1	1	1
2	5	• 2.5
3	8	2.667
4	10	2.25
5	13	2
6	17	**2.833**
7	18	2.428
8	22	• 2.5

Approach 2

Take the ratio of p_i / i, and take the length i whose ratio is maximum.
If there are multiple such i, pick the i with the least value.

Rod length	Price P_i	Revenue (max)
1	1	1
2	5	5
3	8	8
4	9	10
5	10	13
6	17	17
7	17	18
8	20	22

Rod length	New P_i	P_i / i
1	1	1
2	2	1
3	8	2.667
4	9	2.25
5	10	2
6	17	2.833
7	17	2.428
8	20	2.5

Rod length	Revenue (max)	P_i / i
1	1	1
2	5	2.5
3	8	2.667
4	10	2.25
5	13	2
6	17	2.833
7	18	2.428
8	22	2.5

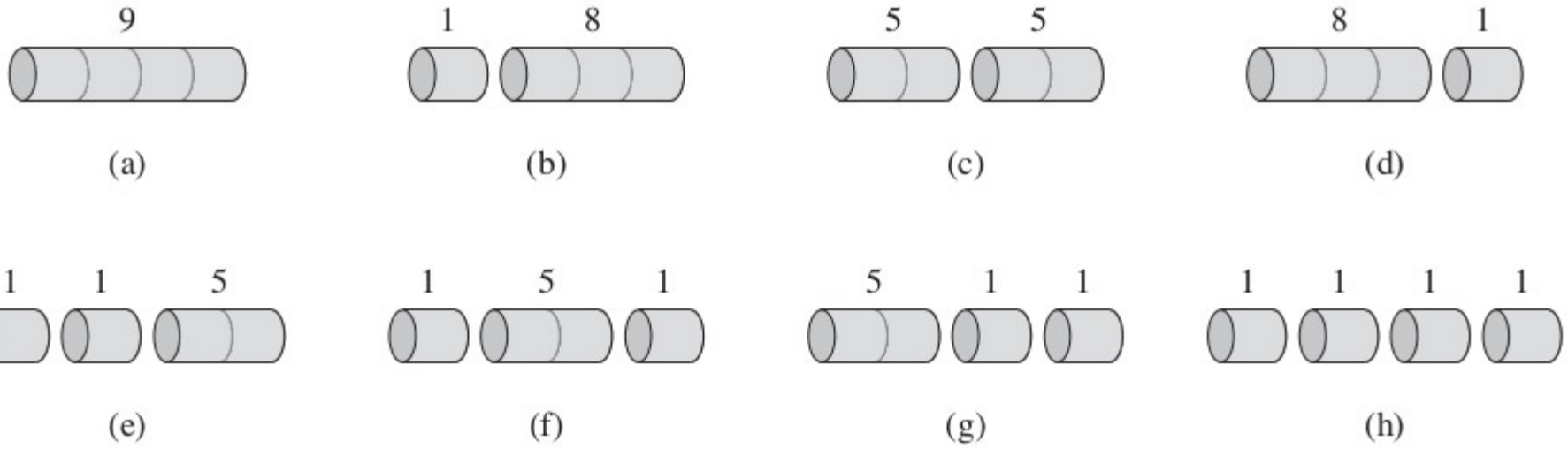
Rod-Cutting Problem

Approach	Time
Brute Force	$O(2^n \cdot n^c)$
Greedy	Does not work
Recursive algorithm	

Intuition?

What is r_n?

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30



Simpler way to decompose

Every optimal solution has a leftmost cut. In other words, there's some cut that gives a first piece of length i cut off the left end, and a remaining piece of length n-i on the right.

- Need to divide only the remainder, not the first piece.
- Leaves only one subproblem to solve, rather than two subproblems.
- Gives a simpler version of the equation for r_n :

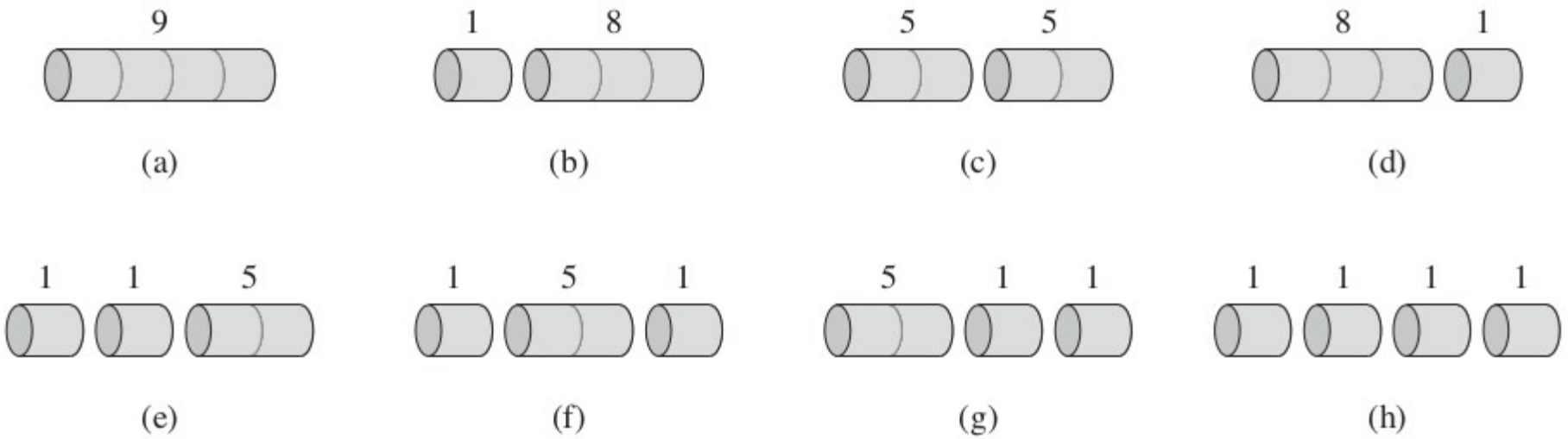
Simpler way to decompose

Every optimal solution has a leftmost cut. In other words, there's some cut that gives a first piece of length i cut off the left end, and a remaining piece of length n-i on the right.

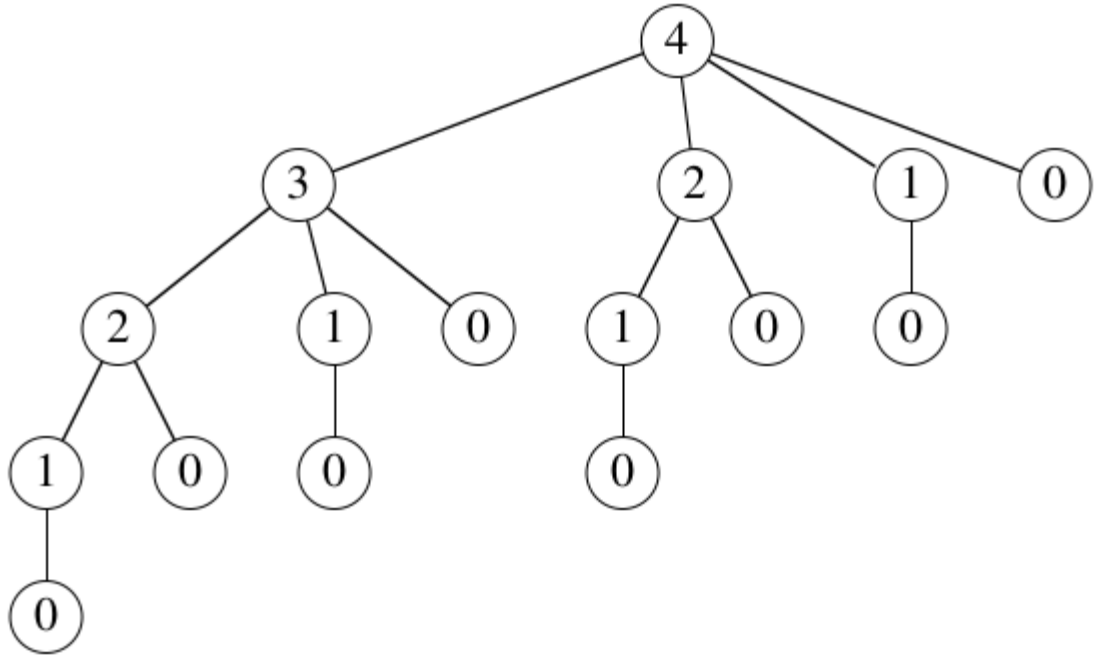
- Need to divide only the remainder, not the first piece.
- Leaves only one subproblem to solve, rather than two subproblems.
- Gives a simpler version of the equation for r_n :

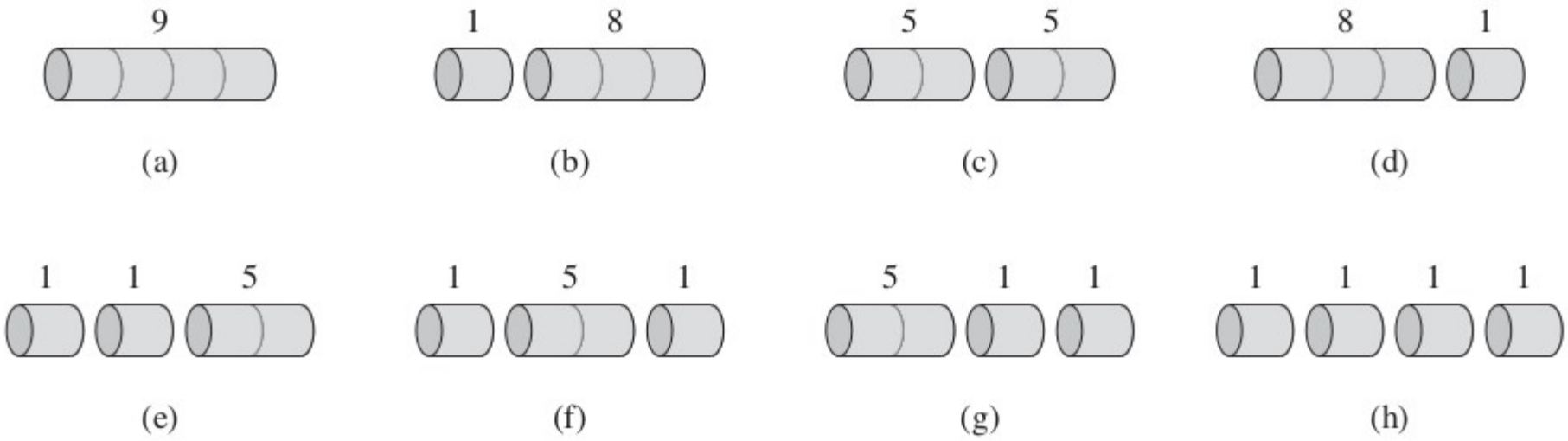
$$r_n = \max\{p_i + r_{n-i} : 1 \le i \le n\}$$

Recursive Tree

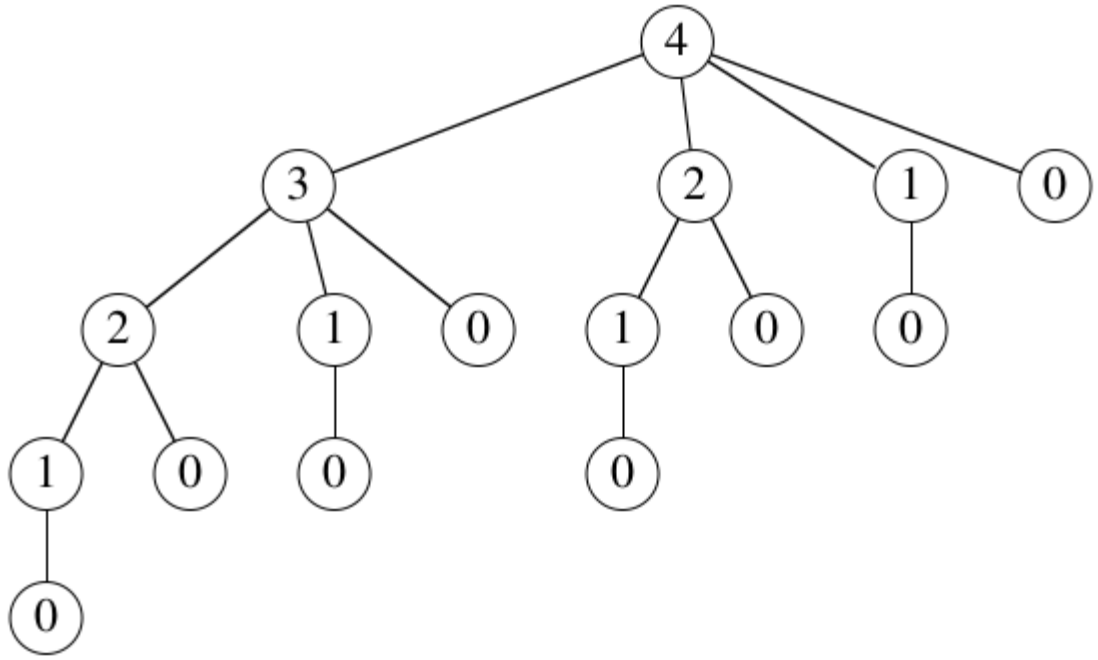


Recursive Tree



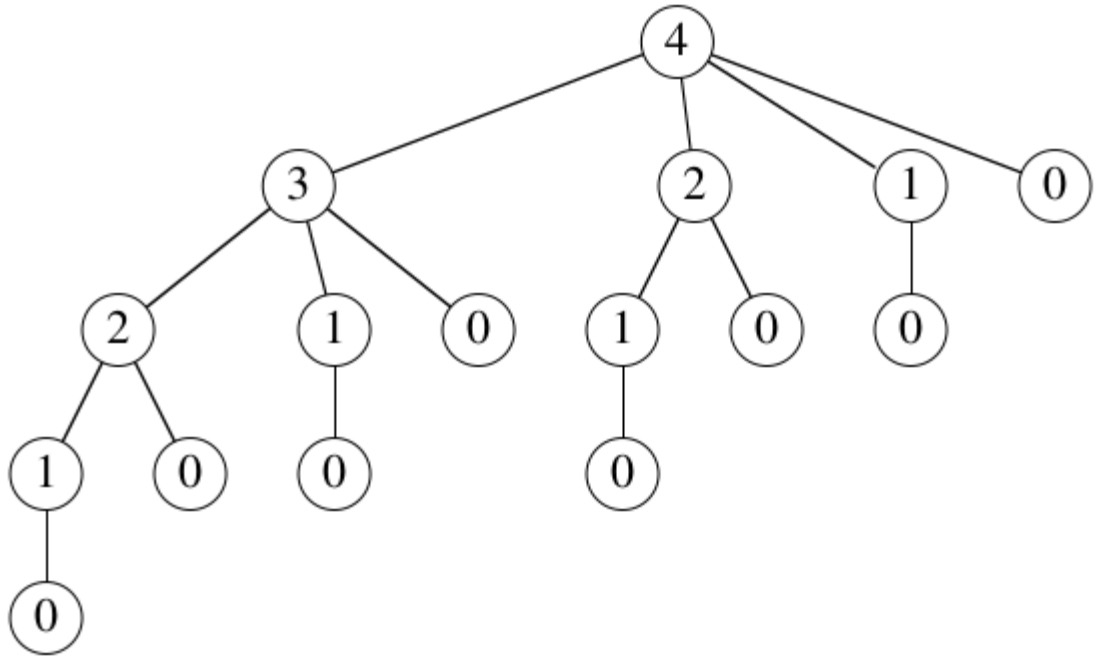


Recursive Function




```
CUT-ROD(p, n)
  if n == 0
      return 0
  q = -∞
  for i = 1 to n
      q = max {q, p[i] + CUT-ROD(p, n - i)}
  return q
```

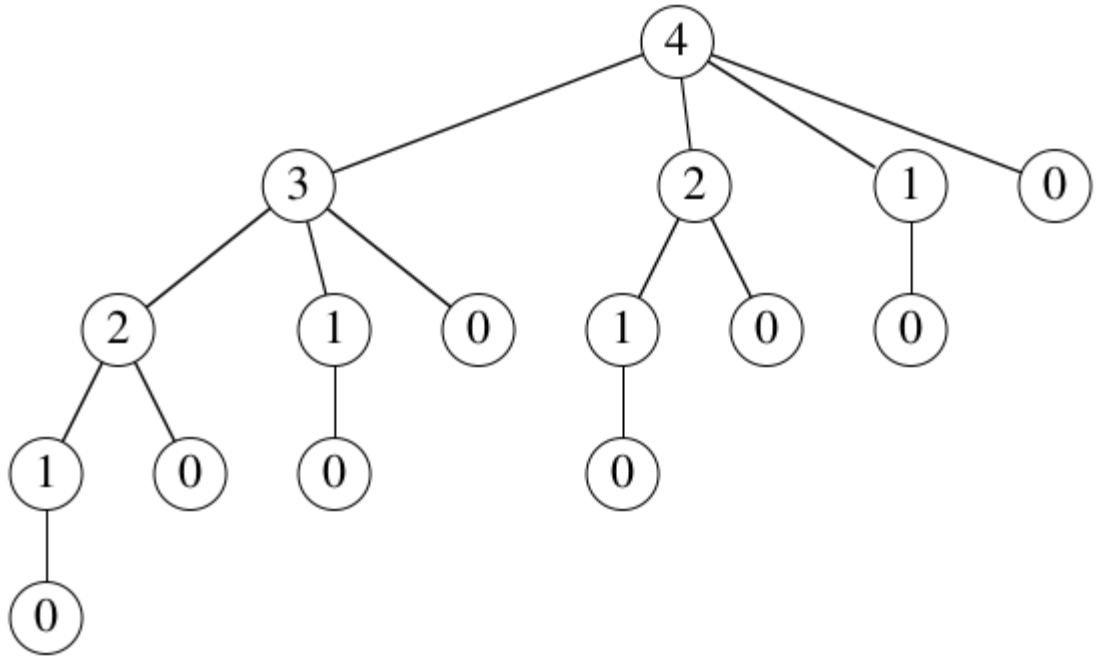
Recurrence?




```
CUT-ROD(p, n)
  if n == 0
      return 0
  q = -∞
  for i = 1 to n
      q = max {q, p[i] + CUT-ROD(p, n - i)}
  return q
```

T(n)=#calls to the function with second parameter n

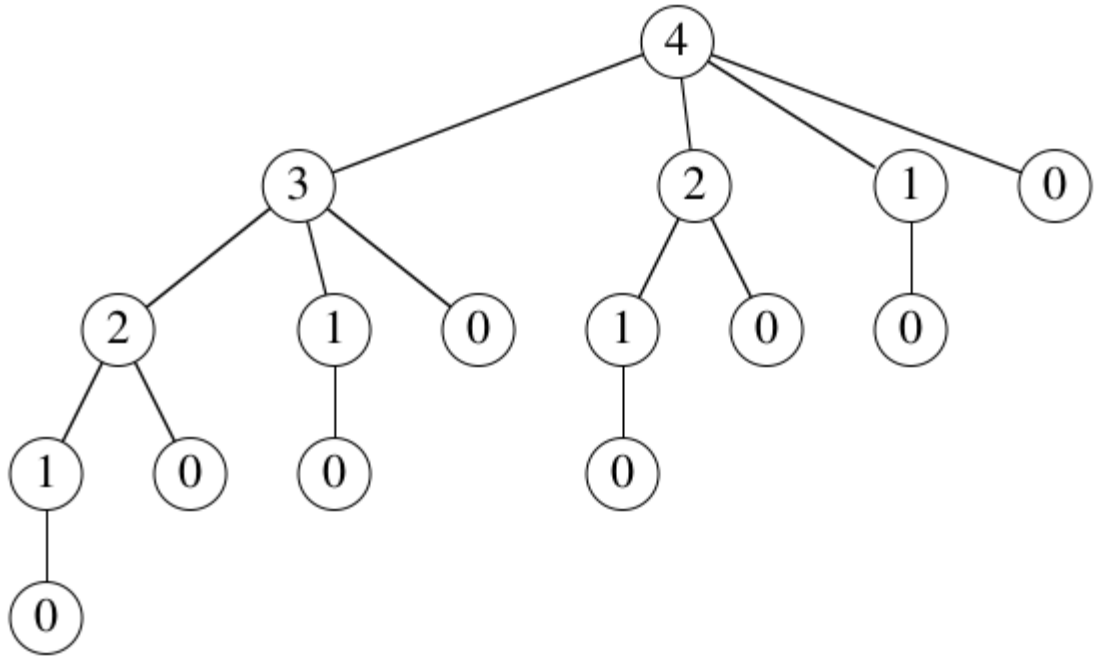
Running time (#recursive calls)?




```
CUT-ROD(p, n)
  if n == 0
      return 0
  q = -∞
  for i = 1 to n
      q = max {q, p[i] + CUT-ROD(p, n - i)}
  return q
```

$$T(n) = \begin{cases} 1 & \text{if } n = 0\,, \\ 1 + \sum_{j=0}^{n-1} T(j) & \text{if } n \geq 1\,. \end{cases}$$

Running time (#recursive calls)?




```
CUT-ROD(p, n)
  if n == 0
      return 0
  q = -∞
  for i = 1 to n
      q = max {q, p[i] + CUT-ROD(p, n - i)}
  return q
```

$O(2^n n)$

$$T(n) = \begin{cases} 1 & \text{if } n = 0 , \\ 1 + \sum_{j=0}^{n-1} T(j) & \text{if } n \geq 1 . \end{cases}$$

Top-down memoization

```
CUT-ROD(p, n)
  if n == 0
      return 0
  q = -∞
  for i = 1 to n
      q = max {q, p[i] + CUT-ROD(p, n - i)}
  return q
```

What to remember?

Top-down memoization

```
MEMOIZED-CUT-ROD(p, n)
  let r[0 : n] be a new array          // will remember solution values in r
  for i = 0 to n
      r[i] = −∞
  return MEMOIZED-CUT-ROD-AUX(p, n, r)
```

Top-down memoization

```
MEMOIZED-CUT-ROD(p, n)
  let r[0:n] be a new array          // will remember solution values in r
  for i = 0 to n
      r[i] = −∞
  return MEMOIZED-CUT-ROD-AUX(p, n, r)
```

```
MEMOIZED-CUT-ROD-AUX(p, n, r)
  if r[n] ≥ 0                        // already have a solution for length n?
      return r[n]
  if n == 0
      q = 0
  else q = −∞
      for i = 1 to n                 // i is the position of the first cut
          q = max{q, p[i] + MEMOIZED-CUT-ROD-AUX(p, n − i, r)}
  r[n] = q                           // remember the solution value for length n
  return q
```

```
CUT-ROD(p, n)
  if n == 0
      return 0
  q = -∞
  for i = 1 to n
      q = max {q, p[i] + CUT-ROD(p, n - i)}
  return q
```

Top-down memoization

```
MEMOIZED-CUT-ROD-AUX(p, n, r)
  if r[n] ≥ 0                    // already have a solution for length n?
      return r[n]
  if n == 0
      q = 0
  else q = -∞
      for i = 1 to n             // i is the position of the first cut
          q = max {q, p[i] + MEMOIZED-CUT-ROD-AUX(p, n - i, r)}
  r[n] = q                       // remember the solution value for length n
  return q
```

Bottom-up ???

Top-down memoization

```
MEMOIZED-CUT-ROD-AUX(p, n, r)
  if r[n] ≥ 0                    // already have a solution for length n?
      return r[n]
  if n == 0
      q = 0
  else q = −∞
      for i = 1 to n             // i is the position of the first cut
          q = max{q, p[i] + MEMOIZED-CUT-ROD-AUX(p, n − i, r)}
  r[n] = q                       // remember the solution value for length n
  return q
```

Bottom-up

```
BOTTOM-UP-CUT-ROD(p, n)
  let r[0 : n] be a new array          // will remember solution values in r
  r[0] = 0
  for j = 1 to n                       // for increasing rod length j
      q = -∞
      for i = 1 to j                   // i is the position of the first cut
          q = max{q, p[i] + r[j - i]}
      r[j] = q                         // remember the solution value for length j
  return r[n]
```

Bottom-up

```
BOTTOM-UP-CUT-ROD(p, n)
  let r[0 : n] be a new array          // will remember solution values in r
  r[0] = 0
  for j = 1 to n                       // for increasing rod length j
      q = −∞
      for i = 1 to j                   // i is the position of the first cut
          q = max{q, p[i] + r[j − i]}
      r[j] = q                         // remember the solution value for length j
  return r[n]
```

Solution?

Reconstructing a solution

```
EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
  let r[0:n] and s[1:n] be new arrays
  r[0] = 0
  for j = 1 to n                    // for increasing rod length j
      q = -∞
      for i = 1 to j                // i is the position of the first cut
          if q < p[i] + r[j - i]
              q = p[i] + r[j - i]
              s[j] = i              // best cut location so far for length j
      r[j] = q                      // remember the solution value for length j
  return r and s
```

Reconstructing a solution

```
PRINT-CUT-ROD-SOLUTION(p, n)
  (r, s) = EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
  while n > 0
      print s[n]          // cut location for length n
      n = n − s[n]        // length of the remainder of the rod
```

Reconstructing a solution

```
PRINT-CUT-ROD-SOLUTION(p, n)
  (r, s) = EXTENDED-BOTTOM-UP-CUT-ROD(p, n)
  while n > 0
      print s[n]          // cut location for length n
      n = n − s[n]        // length of the remainder of the rod
```

Example: For the example, EXTENDED-BOTTOM-UP-CUT-ROD returns

i	0	1	2	3	4	5	6	7	8
$r[i]$	0	1	5	8	10	13	17	18	22
$s[i]$		1	2	3	2	2	6	1	2

Running time

$$\Theta(n^2)$$

Bottom up: Two for loops

Top-down

- Solves each subproblem once each of sizes 0,1,2, …, n
- To Solve a subproblem of size n => for loop iterated n times over all recursive calls.

Subproblem graphs

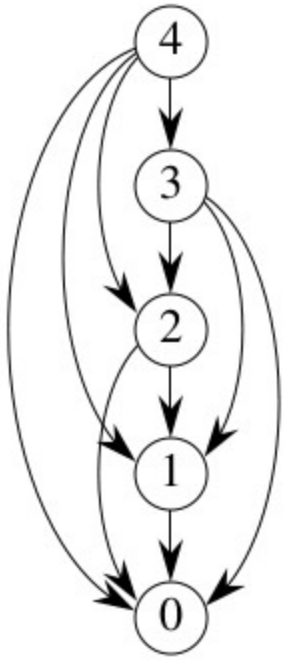
In DP algorithms, you need to understand

- ✓ the set of subproblems involved and
- ✓ how subproblems depend on one another.

This is encoded using **subproblem graphs**.

Helps in determining running time.

Subproblem graphs



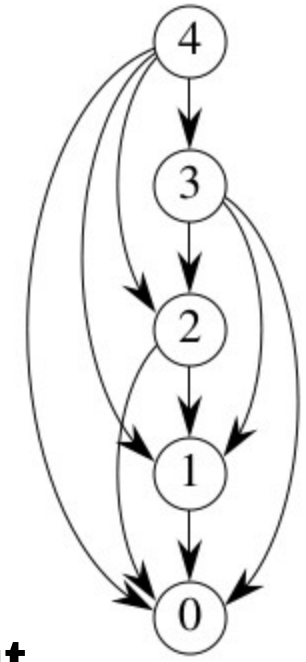
Directed graph:

One vertex for each distinct subproblem.

Has a directed edge (x, y) if computing an optimal solution to subproblem **x** directly requires knowing an optimal solution to subproblem **y**.

Subproblem graph for ROD-CUTTING on n=4

Running time



Because each subproblem is solved just once,

running time is sum of times needed to solve each subproblem.

Time to compute solution to a subproblem is typically linear in the **out-degree (number of outgoing edges) of its vertex**.

Number of subproblems equals **number of vertices**.

When these conditions hold, running time is linear in number of vertices and edges.